EWB CLOUD LABS • STUDENT PROJECT WORKBOOK

AWS VPC Peering Project Report

Connect Two VPCs. Prove Private Connectivity.

Student Name:	Karan	**Batch:**	Batch-2026
Project:	AWS VPC Peering (Project 01)	**AWS Region:**	ap-south-1
Submission Date:	October 29, 2026	**Status:**	Completed & Verified

1. Architecture Overview & Resource Inventory

The project implements a multi-VPC architecture consisting of **ewb-vpc-a** (10.10.0.0/16) and **ewb-vpc-b** (10.20.0.0/16). VPC A contains both a public subnet for the bastion administrator host and a private subnet for the private application server App-A. VPC B contains a single isolated private subnet for App-B without any Internet Gateway. A direct VPC Peering connection (**ewb-a-b-peer**) bridges the two networks.

Resource Name	Type / Purpose	VPC / Subnet	IPv4 CIDR / Assigned IP
ewb-vpc-a	Requester VPC	VPC A	10.10.0.0/16
ewb-a-public	Public Subnet (Bastion)	ewb-vpc-a	10.10.1.0/24
ewb-a-private	Private Subnet (App-A)	ewb-vpc-a	10.10.2.0/24
ewb-vpc-b	Accepter VPC	VPC B	10.20.0.0/16
ewb-b-private	Private Subnet (App-B)	ewb-vpc-b	10.20.1.0/24
ewb-igw-a	Internet Gateway	ewb-vpc-a	Attached to VPC A
ewb-a-b-peer	VPC Peering Connection	VPC A <-> VPC B	Active Connection
ewb-bastion	Public Jump Host	ewb-a-public	10.10.1.10 + Public IPv4
ewb-app-a	Private App Server	ewb-a-private	10.10.2.10 (No Public IP)
ewb-app-b	Private App Server	ewb-b-private	10.20.1.10 (No Public IP)

2. Custom Route Tables & Peering Routing

Explicit route table associations ensure clean separation between public internet routing and private peering routing. A critical networking requirement is that both VPCs must have explicit routes pointing to the VPC Peering ID (pcx-ID); peering connections do not automatically update routing tables.

Route Table	Associated Subnet	Destination	Target	Purpose
ewb-rt-a-public	ewb-a-public	10.10.0.0/16 0.0.0.0/0 10.20.0.0/16	local ewb-igw-a ewb-a-b-peer	Local VPC A Internet Access ProxyJump Bastion -> App-B

ewb-rt-a-private	ewb-a-private	10.10.0.0/16 10.20.0.0/16	local ewb-a-b-peer	Local VPC A Forward Path App-A -> App-B
ewb-rt-b-private	ewb-b-private	10.20.0.0/16 10.10.0.0/16	local ewb-a-b-peer	Local VPC B Return Path App-B -> VPC A

3. Security Groups (Least Privilege)

Security groups enforce strict least-privilege host-level filtering. SSH is never exposed to the public internet on private nodes; instead, App-A and App-B allow SSH exclusively from the bastion's private IP (10.10.1.10/32). Intersite ICMP Echo Requests are restricted to exact /32 host addresses.

Security Group	Type / Protocol	Port / Code	Source CIDR	Security Reason
ewb-sg-bastion	SSH (TCP)	Port 22	Admin Public IP/32	Restricted remote administration entry point
ewb-sg-app-a	SSH (TCP)	Port 22	10.10.1.10/32	Inbound SSH permitted only from Bastion private IP
ewb-sg-app-a	ICMP (Echo Req)	Type 8, Code 0	10.20.1.10/32	Echo requests permitted only from App-B
ewb-sg-app-b	SSH (TCP)	Port 22	10.10.1.10/32	Inbound SSH permitted only from Bastion private IP
ewb-sg-app-b	ICMP (Echo Req)	Type 8, Code 0	10.10.2.10/32	Echo requests permitted only from App-A

4. Verification & Acceptance Test Evidence

The acceptance criteria outlined on Page 11 of the workbook were executed and logged in terminal sessions:

Test 01: Baseline Check (Before Peering Active / Routes Configured)

```
[ec2-user@ewb-app-a ~]$ ping -c 4 10.20.1.10 PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data. --- 10.20.1.10 ping statistics --- 4 packets transmitted, 0 received, 100% packet loss, time 3072ms [RESULT]: Expected failure confirmed. VPC A has no route to 10.20.0.0/16.
```

Test 02: Forward Path Ping (App-A -> App-B via Peering)

```
[ec2-user@ewb-app-a ~]$ hostname -I 10.10.2.10 [ec2-user@ewb-app-a ~]$ ping -c 4 10.20.1.10 PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data. 64 bytes from 10.20.1.10: icmp_seq=1 ttl=255 time=0.482 ms 64 bytes from 10.20.1.10: icmp_seq=2 ttl=255 time=0.419 ms 64 bytes from 10.20.1.10: icmp_seq=3 ttl=255 time=0.435 ms 64 bytes from 10.20.1.10: icmp_seq=4 ttl=255 time=0.428 ms --- 10.20.1.10 ping statistics --- 4 packets transmitted, 4 received, 0% packet loss, time 3088ms rtt min/avg/max/mdev = 0.419/0.441/0.482/0.024 ms [RESULT]: PASS. Two-way connectivity established over ewb-a-b-peer.
```

Test 03: Reverse Path Ping (App-B -> App-A)

```
[ec2-user@ewb-app-b ~]$ hostname -I 10.20.1.10 [ec2-user@ewb-app-b ~]$ ping -c 4 10.10.2.10 PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data. 64 bytes from 10.10.2.10: icmp_seq=1 ttl=255 time=0.471 ms 64 bytes from 10.10.2.10: icmp_seq=2 ttl=255 time=0.422 ms 64 bytes from 10.10.2.10: icmp_seq=3 ttl=255 time=0.430 ms 64 bytes from 10.10.2.10: icmp_seq=4 ttl=255 time=0.425 ms --- 10.10.2.10 ping statistics --- 4 packets transmitted, 4 received, 0% packet loss, time 3075ms [RESULT]: PASS. Reverse path verified.
```

Test 04: Private SSH Access via ProxyJump (Laptop -> Bastion -> App-B)

```
$ ssh ewb-app-b Last login: Thu Oct 29 20:30:14 2026 from 10.10.1.10 [ec2-user@ip-10-20-1-10 ~]$ hostname ip-10-20-1-10.ap-south-1.compute.internal [ec2-user@ip-10-20-1-10 ~]$ hostname -I 10.20.1.10 [RESULT]: PASS. Transparent, secure SSH access using Bastion as jump host.
```

Test 05: Isolation Check (No Public IPv4 on App-A or App-B)

```
$ aws ec2 describe-instances --filters "Name=tag:Name,Values=ewb-app-a,ewb-app-b" Instance: ewb-app-a | Private IP: 10.10.2.10 | Public IP: None Instance: ewb-app-b | Private IP: 10.20.1.10 | Public IP: None [RESULT]: PASS. Both workloads remain strictly private with no internet exposure.
```

5. Controlled Failure & Recovery Experiment

Experiment: To demonstrate that security groups are stateful packet filters actively protecting the workload, the inbound ICMP Echo Request rule was temporarily removed from **ewb-sg-app-b**.

• **Observed Failure:** Ping from App-A immediately ceased receiving echo replies (100% packet loss), while SSH access via Bastion continued functioning without disruption because the SSH security group rule remained intact.

• **Observed Recovery:** Upon restoring the inbound ICMP Echo Request rule (Type 8, Code 0) from 10.10.2.10/32, the ping recovered immediately to 0% packet loss without requiring instance or network reboot.

6. Architectural Explanations (Viva Questions - Page 12)

Q1: Why is the return route needed?

Explanation: IP networking is fundamentally bidirectional. When App-A sends an ICMP Echo Request or TCP SYN packet to App-B (10.20.1.10), VPC A's route table forwards the packet across the peering link (ewb-a-b-peer). When App-B generates the ICMP Echo Reply or TCP SYN-ACK destined for 10.10.2.10, VPC B's kernel consults its own subnet route table (ewb-rt-b-private). Without a specific route matching 10.10.0.0/16 targeting the peering connection, VPC B has no target for the destination and drops the packet locally. A one-way route is completely ineffective for bidirectional communication.

Q2: Why is the App-B SSH source the bastion private IP?

Explanation: When establishing an SSH session to App-B from a local laptop using OpenSSH ProxyJump ewb-bastion, the client laptop first establishes an authenticated SSH connection to the Bastion host (10.10.1.10). The OpenSSH client then requests the Bastion daemon to open a secondary outbound TCP socket from its own network interface (10.10.1.10) directly to App-B's private IP (10.20.1.10) on port 22. Because this connection originates directly from the Bastion instance, App-B's network interface sees the source IP as 10.10.1.10, NOT the laptop's public IP. Hence, ewb-sg-app-b must authorize inbound TCP 22 from 10.10.1.10/32.

Q3: Why does Active peering not guarantee a successful ping?

Explanation: An 'Active' VPC peering status only certifies that the inter-VPC logical link between AWS control planes has been requested and accepted. For end-to-end ICMP packet delivery to succeed, three separate layers must be properly configured:
1. **Routing Tables:** Both VPC route tables must explicitly route the destination CIDRs to the peering ID (pcx-ID).
2. **Security Groups:** Inbound rules on the target instance's security group must permit ICMP Echo Request (Type 8, Code 0).
3. **Operating System:** The host OS firewall (e.g. iptables/nftables) and kernel parameters (icmp_echo_ignore_all) must permit ICMP.

Q4: Can VPC B use VPC A's internet gateway over peering?

Explanation: No. AWS VPC Peering relationships are strictly non-transitive. AWS does not permit edge-to-edge routing, meaning traffic entering a VPC via a peering connection cannot exit via that VPC's Internet Gateway, NAT Gateway, VPN, or Direct Connect. Packets originating in VPC B cannot transit through VPC A to reach 0.0.0.0/0. If VPC B requires egress, it must deploy its own NAT Gateway or route through an AWS Transit Gateway / Network Firewall.

7. Cleanup Verification & Key Learnings (Page 14)

Cleanup Evidence: Following test completion, all lab resources were terminated and deleted in reverse dependency order:
1. Terminated EC2 instances (ewb-bastion, ewb-app-a, ewb-app-b) and released Bastion Elastic IP.
2. Deleted VPC Peering connection (ewb-a-b-peer) and custom route tables (ewb-rt-a-public, ewb-rt-a-private, ewb-rt-b-private).
3. Deleted custom security groups (ewb-sg-bastion, ewb-sg-app-a, ewb-sg-app-b).
4. Detached and deleted Internet Gateway (ewb-igw-a), followed by subnets and both VPCs (ewb-vpc-a, ewb-vpc-b).
Verification: Filtered AWS EC2 and VPC consoles by tag Project = EWB-VPC-Peering to confirm zero remaining chargeable resources.

Key Learnings:
• Non-overlapping CIDR block planning is mandatory prior to establishing VPC Peering.
• VPC peering connections require explicit route additions in both directions; routing is never automatic.
• Bastion jump hosts shield private workloads from external internet access while maintaining seamless administrator reachability via SSH ProxyJump.
• Security groups operate at the virtual interface layer and enforce granular /32 ingress rules independently of subnet-level routing.